DOC 06 · CAPATAZ AI

Arquitectura Funcional del Producto

Dashboard Operativo · ultra-plus blueprint

Definición de piezas, responsabilidades y evolución del sistema. Sin código todavía: pura arquitectura con dientes, preparada para MVP y para la expansión futura a score avanzado, reportes serios, alertas multicanal y WhatsApp.

Documento: 06_Arquitectura_Funcional_Capataz	Versión: 1.0 · Marzo 2026
Base: Product Vision + PRD + Personas/Roles + User Stories + Flujos	Objetivo: decidir piezas y stack sin entrar a código

1. Tesis arquitectónica

Capataz Dashboard debe comportarse como un sistema de mando operativo en tiempo real, no como una web bonita con tablas. La arquitectura propuesta parte del stack base del PRD y lo lleva a una versión ultra-plus: una base sobria para entregar el MVP sin drama, pero con las costuras ya preparadas para score avanzado, alertas multicanal, WhatsApp y capas de IA operativa.

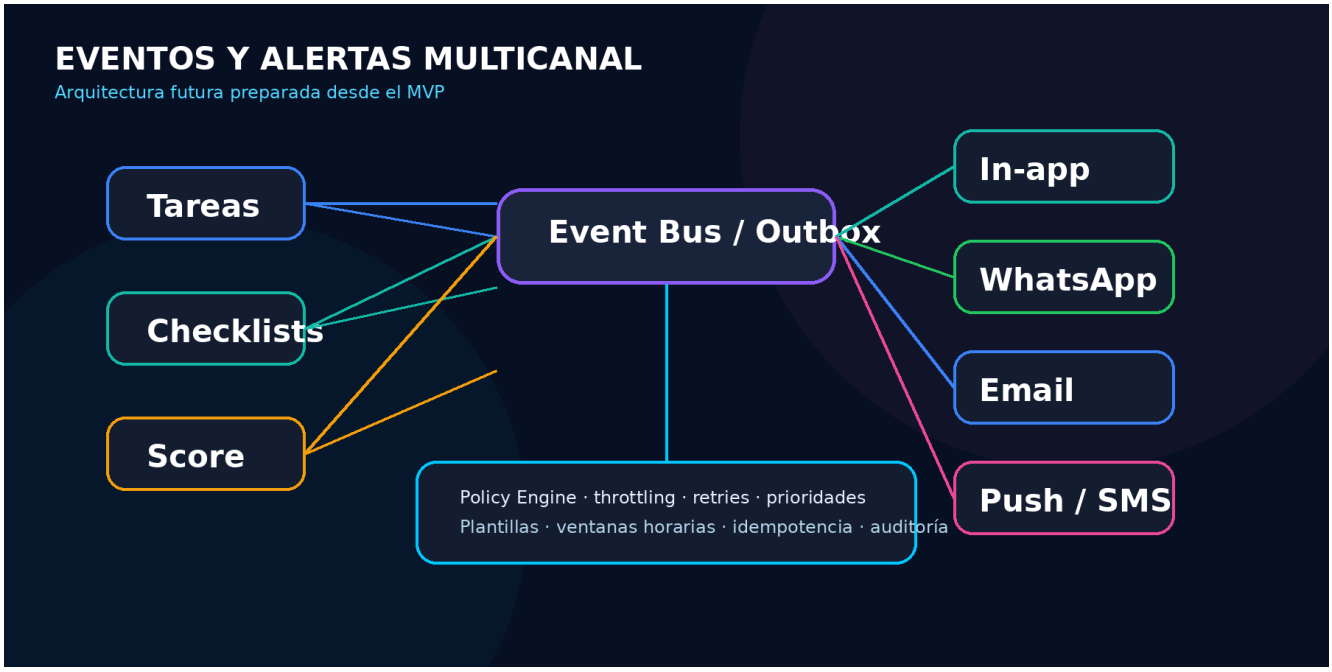
Meta: entregar el MVP del dashboard con un monolito modular premium, pero sembrando desde el día uno la separación de dominio, el outbox de eventos y los adaptadores para canales futuros.

2. Principios rectores

- Operaciones primero: cada decisión técnica debe reducir fricción en tareas, checklists, score y alertas.
- Reactividad sin caos: tiempo real donde aporta valor, no en cada pixel por deporte.
- Seguridad desde el dato: el control por rol vive en auth, backend y base de datos, no solo en el frontend.
- Dominio desacoplado: tareas, score, reportes y notificaciones evolucionan como piezas separadas.
- Compatibilidad con futuro multicanal: toda señal operativa importante debe emitirse como evento, no solo como cambio local.
- WhatsApp-ready desde hoy: aunque no entre al MVP, la arquitectura deja preparado el adaptador.

3. Vista macro del sistema

Capataz se organiza en capas: experiencia, orquestación de aplicación, servicios de dominio, datos/eventos y servicios externos. El diseño evita que el frontend se convierta en cerebro oculto del producto y deja a la base de datos como sistema de registro auditable.


EVENTOS Y ALERTAS MULTICANAL
Arquitectura futura preparada desde el MVP
Tareas
Checklists
Score
Event Bus / Outbox
In-app
WhatsApp
Email
Push / SMS
Policy Engine · throttling · retries · prioridades
Plantillas · ventanas horarias · idempotencia · auditoría

4. Frontend

Un front premium, rápido y expresivo, con el lenguaje visual de LotOS UI encima de una arquitectura seria.

Recomendación / stack	Responsabilidad / evolución
• Next.js App Router + React + TypeScript estricto + Tailwind CSS + sistema de diseño Capataz/LotOS UI. Complementos recomendados: React Hook Form, Zod, dnd-kit, Zustand para estado efímero y TanStack Table para vistas complejas. • LotOS UI como capa estética superior: tokens, radios amplios, contraste fuerte, tarjetas operativas, badges y microinteracciones sobrias. • Views core: Home, Kanban, Tareas, Equipo, Score, Reportes, Checklists, Alertas y Config. • Estado local efímero en cliente; estado de negocio y mutaciones verdaderas viven del lado servidor.	• Renderizar Home, Kanban, Tareas, Equipo, Score, Reportes, Checklists, Alertas y Config. • Ejecutar validaciones de interacción y UX inmediata. • Consumir Server Actions para mutaciones cercanas a la UI y endpoints dedicados para procesos largos o integraciones. • Escuchar eventos de tiempo real para refrescar métricas, tarjetas, alertas y score sin recarga manual. • MVP: Dashboard web responsive, foco en desktop y tablet, con móvil operativo para supervisores y operadores. • Futuro: Portal conversacional de supervisión, kiosco ligero para sucursal y app móvil dedicada si el volumen lo exige.

5. Backend

Un backend con forma de BFF al inicio y con dominios claros para no convertirse en sopa de endpoints.

API Routes + Server Actions como capa de aplicación en Next.js para el MVP. Recomendado preparar workers dedicados para reporting, scoring y notificaciones. Si el throughput crece, esos workers pueden migrar a Go o Node aislado sin tocar el front.

Componente	Responsabilidad
BFF / App Layer	Autorización, composición de payloads, validación de inputs, orquestación de consultas y mutaciones.
Task Service	CRUD de tareas, transición de estados, asignaciones, historial y vistas Kanban/lista.
Checklist Service	Plantillas, instancias, avance, cierre y auditoría de cumplimiento.
Score Service	Cálculo determinista, agregación de métricas y publicación de score actualizado.
Report Service	Generación de snapshots, PDFs y vistas analíticas por rango.
Notification Service	Alertas internas hoy; adaptadores multicanal mañana.
Integration Service	Webhooks, conectores externos y futura integración con Meta Cloud API.

MVP: La capa principal puede vivir dentro del monolito modular de Next.js para velocidad de entrega.

Futuro: Separar por servicios o workers cuando haya más volumen, clientes y canales, no antes.

6. Auth

Usar Supabase Auth como recomendación principal para el MVP y como identidad base del producto.

Recomendación / stack	Responsabilidad / evolución
• Se integra nativamente con PostgreSQL, RLS y sesiones del dashboard. • Reduce fricción de implementación en Vercel + Supabase. • Permite modelar RBAC real con claims, tablas de roles y políticas de base de datos. • NextAuth solo tendría ventaja si más adelante se prioriza SSO empresarial amplio o múltiples proveedores externos.	• Autenticación por email/password para MVP. • Sesión persistente segura con refresh controlado. • Resolución de rol efectiva: admin, dueño, supervisor, operador. • Permisos de pantalla y acción derivados de rol + pertenencia de datos. • Futuro: Agregar MFA para admins, magic link opcional, SSO empresarial y sesiones de dispositivo para auditoría.

7. Base de datos

PostgreSQL/Supabase como sistema de registro único, con seguridad al nivel de fila y vistas de lectura optimizadas.

Tablas y agregados sugeridos:

Dominios / tablas núcleo
users, organizations, memberships, roles
tasks, task_statuses, task_assignments, task_events
checklist_templates, checklist_instances, checklist_items
scores, score_snapshots, score_factors
alerts, alert_rules, notifications, notification_deliveries
reports, report_jobs, report_artifacts
audit_log, integration_events, webhook_deliveries

Patrones de diseño de datos:

- Row Level Security para aislamiento por rol y por organización.
- Vistas y materialized views para Home, ranking de score y reportes rápidos.
- Outbox table para publicar eventos sin perder consistencia transaccional.
- Soft delete y auditoría en lugar de borrado duro para entidades operativas.

Futuro: Reservar espacio para pgvector y documentos semiestructurados si la capa de IA contextual gana peso.

8. Tiempo real

Supabase Realtime para el MVP. Event bus + websockets especializados como evolución natural.

Recomendación / stack	Responsabilidad / evolución
• Actualización de tarjetas Kanban al mover estados. • Refresco de KPIs de Home cuando se crea o completa una tarea. • Entrada de alertas internas sin recarga. • Actualización del score visible al cerrar tareas o checklists.	• Cuando entren notificaciones multicanal y mayor concurrencia, introducir una capa de eventos con outbox + broker. El websocket del dashboard se vuelve un consumidor más, no el centro del universo. • Regla práctica: usar realtime para visibilidad operativa y coordinación, no para reemplazar toda la lectura transaccional.

9. Generación de reportes

Separar claramente analytics visuales de exportación formal.

Recomendación / stack	Responsabilidad / evolución
• Para gráficas del dashboard, usar Chart.js o ECharts según preferencia visual. Si se busca impacto visual premium, ECharts gana; si se prioriza velocidad del MVP, Chart.js es suficiente. • Para PDFs serios, la recomendación premium es generación server-side a partir de plantillas HTML renderizadas por Chromium/Playwright. jsPDF queda como opción rápida para exportación cliente en MVP, pero no como techo final.	• Consultar vistas agregadas por rango de fechas. • Renderizar dashboard analítico en servidor. • Generar PDF y guardar artefacto con metadatos y timestamp. • Dejar el snapshot disponible para descarga y auditoría.

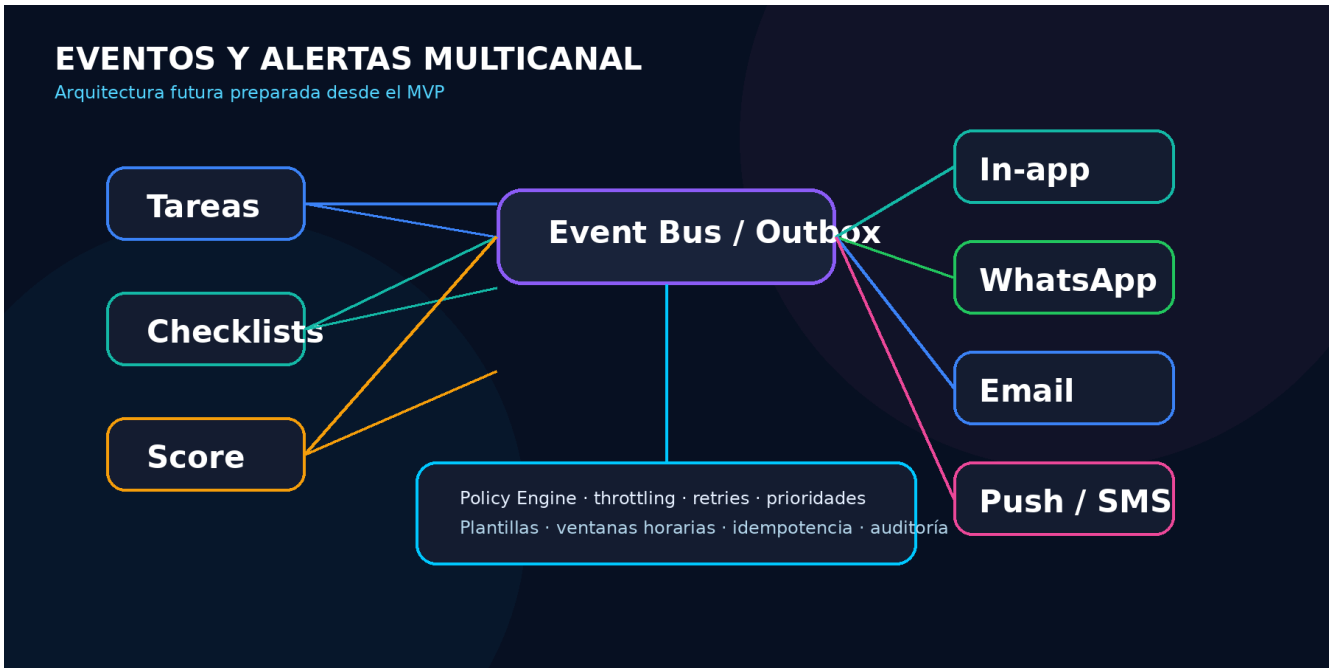
10. Motor de score

El score base no debe depender de IA. Debe ser determinista, explicable y auditable.

Recomendación / stack	Responsabilidad / evolución
• Cumplimiento de tareas asignadas. • Velocidad relativa contra fecha límite. • Consistencia en periodos diarios/semanales. • Cumplimiento de checklists. • Incidencias por vencimiento, retraso o reasignación.	• Eventos operativos entran al Score Service. • El servicio recalcula factores afectados, no todo el universo. • Se guarda score actual y snapshots por periodo. • El dashboard consulta tanto score actual como desglose de factores. • Futuro: Encima del score determinista se puede sumar una capa IA que explique tendencias, recomiende acciones y detecte anomalías. La IA interpreta; la base no inventa.

11. Motor futuro de alertas multicanal

Las alertas deben evolucionar de panel interno a red de notificación con políticas, prioridades y auditoría.



- Cada evento relevante genera una intención de notificación.
- Un Policy Engine decide canal, prioridad, horario, plantilla y reintentos.
- Los Channel Adapters despachan a in-app, WhatsApp, email, push o SMS.
- Un Delivery Log guarda estado, latencia, error y confirmación por canal.

MVP: Solo alertas in-app visibles en dashboard y sidebar.

Futuro: Activar reglas como: tarea vencida = in-app inmediato, WhatsApp al operador, resumen email al supervisor y escalamiento al dueño si no hay respuesta.

12. Integración futura con WhatsApp

WhatsApp no debe entrar como parche. Debe ser un adaptador formal del dominio operativo.

Pieza	Rol en la arquitectura
Webhook Inbound	recibe mensajes, respuestas y estados de Meta Cloud API.
Message Orchestrator	traduce eventos del dominio a mensajes salientes.
Template Manager	administra plantillas aprobadas, variables y versiones.
Conversation Resolver	asocia mensajes a usuario, tarea, checklist o alerta.
Delivery Tracker	guarda idempotencia, estados y reintentos.

Casos de uso previstos:

- Asignación de tarea con botón o respuesta rápida.
- Recordatorios de checklist de apertura/cierre.
- Confirmación de tarea completada desde WhatsApp.
- Escalamiento de alertas urgentes fuera del dashboard.

Guardrails:

- No depender de WhatsApp para la lógica crítica del sistema.
- Todo mensaje debe existir también como evento interno auditable.
- Las acciones entrantes deben validarse contra identidad, rol y contexto.

13. Lenguajes y runtimes recomendados

La arquitectura no está casada con una sola espada, pero sí con una disciplina clara.

Recomendación / stack	Responsabilidad / evolución
• TypeScript como lenguaje principal de producto web. • SQL serio para reglas, vistas y agregaciones donde conviene. • Go opcional para workers de alto volumen o sockets si el sistema crece agresivamente. • Python opcional para analítica, IA contextual y experimentos de scoring avanzado.	• La clave no es presumir lenguajes como espada de anime. La clave es dejar cada cosa donde rinde más.

14. Despliegue e infraestructura

MVP: Vercel para frontend/BFF, Supabase para PostgreSQL/Auth/Realtime/Storage.

- Ambientes dev, preview y producción.
- Feature flags para piezas futuras como columnas configurables o canales externos.
- Logs estructurados, tracing básico y alertas de errores.
- Backups, migraciones versionadas y seed data idempotente para demo y QA.

Futuro: Workers asíncronos en contenedores o funciones separadas para score pesado, PDF batch y notificaciones.

15. Roadmap de evolución

Fase	Qué cambia
Fase 1 · MVP Domingo	Monolito modular en Next.js + Supabase, realtime nativo, score determinista, PDFs básicos, alertas in-app.
Fase 2 · Operación seria	Outbox, workers, PDFs server-side, score snapshot por periodo, auditoría más fina.
Fase 3 · Omnicanal	Policy Engine, adapters por canal, Meta Cloud API, email/push, trazabilidad completa.
Fase 4 · Inteligencia operativa	IA contextual, resumen automático, detección de anomalías, copiloto de supervisión.

16. Anti-patrones a evitar

- No meter toda la lógica en componentes del frontend.

- No usar el score como caja negra imposible de explicar.
- No resolver permisos solo ocultando botones.
- No acoplar WhatsApp directamente a tablas críticas sin capa de integración.
- No generar PDFs complejos solo del lado cliente si van a compartirse formalmente.

17. Recomendación final

La recomendación para Capataz es arrancar con un monolito modular premium: Next.js + Supabase + Tailwind/LotOS UI + Server Actions + Realtime, pero diseñando desde el día uno con dominios separados, outbox de eventos y un servicio claro de score/notificaciones. Así el MVP nace ligero, pero no nace chiquito.

En otras palabras: un chasis de entrega rápida, con motor de expansión real. La criatura no solo camina el domingo; queda lista para rugir cuando entren WhatsApp, omnicanalidad, IA operativa y crecimiento multi-sucursal.